

kickstart_lecture3_numpy

May 26, 2025

0.1 Python Kickstart Lecture 3: First Steps with Numpy

0.1.1 Importing Libraries

```
[3]: import numpy as np
```

0.1.2 Creating arrays

```
[5]: # 1D array
array_1D = np.array([5,6.4,7,8.9])
```

```
[6]: array_1D
```

```
[6]: array([5. , 6.4, 7. , 8.9])
```

```
[7]: type(array_1D)
```

```
[7]: numpy.ndarray
```

```
[8]: # 2D array
array_2D = np.array([[1,2,3],[11,22,33],[111,222,333]])
```

```
[9]: array_2D
```

```
[9]: array([[ 1,  2,  3],
          [11, 22, 33],
          [111, 222, 333]])
```

```
[10]: # 3D array
array_3D = np.array([[[1,2],[3,4]],[[11,22],[33,44]]])
```

```
[11]: array_3D
```

```
[11]: array([[[ 1,  2],
            [ 3,  4]],

           [[11, 22],
            [33, 44]]])
```

```
[12]: # len function
      len(array_1D)
```

```
[12]: 4
```

```
[13]: len(array_3D)
```

```
[13]: 2
```

```
[14]: # .size attribute
      array_3D.size
```

```
[14]: 8
```

```
[15]: # .shape attribute
      array_3D.shape
```

```
[15]: (2, 2, 2)
```

0.1.3 Accessing Elements from Arrays

```
[17]: array_1D
```

```
[17]: array([5. , 6.4, 7. , 8.9])
```

```
[18]: # Accessing one element from 1D array
      array_1D[2]
```

```
[18]: 7.0
```

```
[19]: # Accessing single elements from 3D arrays
      array_3D
```

```
[19]: array([[[ 1,  2],
            [ 3,  4]],

          [[11, 22],
            [33, 44]]])
```

```
[20]: array_3D[1,0,1]
```

```
[20]: 22
```

```
[21]: # Accessing multiple elements: slicing
      array_1D[1:3]
```

```
[21]: array([6.4, 7. ])
```

```
[ ]:
```

0.1.4 Array Handling: Concatenating and Reshaping

```
[23]: # First create 3 1D-arrays
```

```
a1 = np.array([1,2,3])  
a2 = np.array([4,5,6])  
a3 = np.array([7,8,9])
```

```
[24]: # And merge them
```

```
array_merged = np.concatenate([a1,a2,a3])  
array_merged
```

```
[24]: array([1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
[25]: # Test reshape
```

```
array_2D_from_1D = array_merged.reshape((3,3))  
array_2D_from_1D
```

```
[25]: array([[1, 2, 3],  
           [4, 5, 6],  
           [7, 8, 9]])
```

```
[26]: # Stacking: vertical and horizontal
```

```
array_vstack = np.vstack((a1,a2,a3))  
array_vstack
```

```
[26]: array([[1, 2, 3],  
           [4, 5, 6],  
           [7, 8, 9]])
```

```
[27]: np.hstack((a1.reshape(3,1),a2.reshape(3,1),a3.reshape(3,1)))
```

```
[27]: array([[1, 4, 7],  
           [2, 5, 8],  
           [3, 6, 9]])
```

```
[28]: # Transpose
```

```
array_vstack.transpose()
```

```
[28]: array([[1, 4, 7],  
           [2, 5, 8],  
           [3, 6, 9]])
```

```
[29]: # Check available methods with dir
```

```
dir(array_1D)
```

```
[29]: ['T',
      '__abs__',
      '__add__',
      '__and__',
      '__array__',
      '__array_finalize__',
      '__array_function__',
      '__array_interface__',
      '__array_prepare__',
      '__array_priority__',
      '__array_struct__',
      '__array_ufunc__',
      '__array_wrap__',
      '__bool__',
      '__buffer__',
      '__class__',
      '__class_getitem__',
      '__complex__',
      '__contains__',
      '__copy__',
      '__deepcopy__',
      '__delattr__',
      '__delitem__',
      '__dir__',
      '__divmod__',
      '__dlpack__',
      '__dlpack_device__',
      '__doc__',
      '__eq__',
      '__float__',
      '__floordiv__',
      '__format__',
      '__ge__',
      '__getattr__',
      '__getitem__',
      '__getstate__',
      '__gt__',
      '__hash__',
      '__iadd__',
      '__iand__',
      '__ifloordiv__',
      '__ilshift__',
      '__imatmul__',
      '__imod__',
      '__imul__',
      '__index__',
      '__init__']
```

```

'__init_subclass__',
'__int__',
'__invert__',
'__ior__',
'__ipow__',
'__irshift__',
'__isub__',
'__iter__',
'__itruediv__',
'__ixor__',
'__le__',
'__len__',
'__lshift__',
'__lt__',
'__matmul__',
'__mod__',
'__mul__',
'__ne__',
'__neg__',
'__new__',
'__or__',
'__pos__',
'__pow__',
'__radd__',
'__rand__',
'__rdivmod__',
'__reduce__',
'__reduce_ex__',
'__repr__',
'__rfloordiv__',
'__rlshift__',
'__rmatmul__',
'__rmod__',
'__rmul__',
'__ror__',
'__rpow__',
'__rrshift__',
'__rshift__',
'__rsub__',
'__rtruediv__',
'__rxor__',
'__setattr__',
'__setitem__',
'__setstate__',
'__sizeof__',
'__str__',
'__sub__',

```

```
'__subclasshook__',  
'__truediv__',  
'__xor__',  
'all',  
'any',  
'argmax',  
'argmin',  
'argpartition',  
'argsort',  
'astype',  
'base',  
'byteswap',  
'choose',  
'clip',  
'compress',  
'conj',  
'conjugate',  
'copy',  
'ctypes',  
'cumprod',  
'cumsum',  
'data',  
'diagonal',  
'dot',  
'dtype',  
'dump',  
'dumps',  
'fill',  
'flags',  
'flat',  
'flatten',  
'getfield',  
'imag',  
'item',  
'itemset',  
'itemsizes',  
'max',  
'mean',  
'min',  
'nbytes',  
'ndim',  
'newbyteorder',  
'nonzero',  
'partition',  
'prod',  
'ptp',  
'put',
```

```
'ravel',
'real',
'repeat',
'reshape',
'resize',
'round',
'searchsorted',
'setfield',
'setflags',
'shape',
'size',
'sort',
'squeeze',
'std',
'strides',
'sum',
'swapaxes',
'take',
'tobytes',
'tofile',
'tolist',
'tostring',
'trace',
'transpose',
'var',
'view']
```

0.1.5 Math Operations

```
[31]: # Element-wise multiplication (use 3 1D-arrays from above)
a1,a2,a3
```

```
[31]: (array([1, 2, 3]), array([4, 5, 6]), array([7, 8, 9]))
```

```
[32]: a1*a2
```

```
[32]: array([ 4, 10, 18])
```

```
[33]: # Flipping row to column and multiply
a1.reshape(-1,1)*a2
```

```
[33]: array([[ 4,  5,  6],
           [ 8, 10, 12],
           [12, 15, 18]])
```

```
[34]: # The same works for division
a1.reshape(-1,1)/a2
```

```
[34]: array([[0.25      , 0.2       , 0.16666667],
            [0.5       , 0.4       , 0.33333333],
            [0.75      , 0.6       , 0.5       ]])
```

```
[35]: a1**a2
```

```
[35]: array([ 1, 32, 729])
```

0.1.6 Helpful Numpy Functions and Methods

```
[37]: # Template arrays with zeros
      np.zeros((5,5))
```

```
[37]: array([[0., 0., 0., 0., 0.],
            [0., 0., 0., 0., 0.],
            [0., 0., 0., 0., 0.],
            [0., 0., 0., 0., 0.],
            [0., 0., 0., 0., 0.]])
```

```
[38]: # ... with ones
      np.ones((2,6))
```

```
[38]: array([[1., 1., 1., 1., 1., 1.],
            [1., 1., 1., 1., 1., 1.]])
```

```
[39]: # arange
      np.arange(1,4,.1)
```

```
[39]: array([1. , 1.1, 1.2, 1.3, 1.4, 1.5, 1.6, 1.7, 1.8, 1.9, 2. , 2.1, 2.2,
            2.3, 2.4, 2.5, 2.6, 2.7, 2.8, 2.9, 3. , 3.1, 3.2, 3.3, 3.4, 3.5,
            3.6, 3.7, 3.8, 3.9])
```

```
[40]: linsp_var = np.linspace(1,4,31)
      linsp_var
```

```
[40]: array([1. , 1.1, 1.2, 1.3, 1.4, 1.5, 1.6, 1.7, 1.8, 1.9, 2. , 2.1, 2.2,
            2.3, 2.4, 2.5, 2.6, 2.7, 2.8, 2.9, 3. , 3.1, 3.2, 3.3, 3.4, 3.5,
            3.6, 3.7, 3.8, 3.9, 4. ])
```

```
[41]: # Statistics: std, max, min
      linsp_var.std()
```

```
[41]: 0.894427190999916
```

```
[42]: linsp_var.min()
```

```
[42]: 1.0
```


[]: